

Getting Started with Agentic QA Platform

Installation

Prerequisites

- Node.js 20 or higher
- Redis (optional, for caching)
- PostgreSQL (optional, for reporting)
- Anthropic API key or OpenAI API key

Quick Setup

```
# Clone or create the project
cd agentic-qa-platform

# Install dependencies
npm install

# Install Playwright browsers
npx playwright install chromium

# Set up environment variables
cp .env.example .env
# Edit .env and add your API keys
```

Configuration

Edit `.env` file:

```
# Required
ANTHROPIC_API_KEY=your_key_here

# Optional
OPENAI_API_KEY=your_openai_key
REDIS_URL=redis://localhost:6379
DATABASE_URL=postgresql://user:pass@localhost:5432/agentic_qa

# Browser settings
HEADLESS=true
BROWSER_TIMEOUT=30000

# Self-healing
ENABLE_SELF_HEALING=true
AUTO_HEAL_CONFIDENCE_THRESHOLD=0.8
REQUIRE_HUMAN_APPROVAL=true
```

Running Tests

Method 1: Natural Language Tests via API

1. Start the API server:

```
npm run start
```

2. Open the web interface:

```
open src/ui/index.html
```

3. Write your test in plain English and click "Execute Test"

Method 2: Playwright Test Integration

Write tests in `tests/` directory:

```
import { test } from './fixtures/agent-test';

test('My natural language test', async ({ agent }) => {
  const result = await agent.execute(`
    1. Navigate to https://example.com
    2. Click login button
    3. Fill username with "test_user"
    4. Fill password with "Test123!"
    5. Click submit
    6. Verify dashboard is displayed
  `);

  expect(result.success).toBeTruthy();
});
```

Run tests:

```
npm test
```

Method 3: Self-Healing Utilities

Use self-healing in traditional Playwright tests:

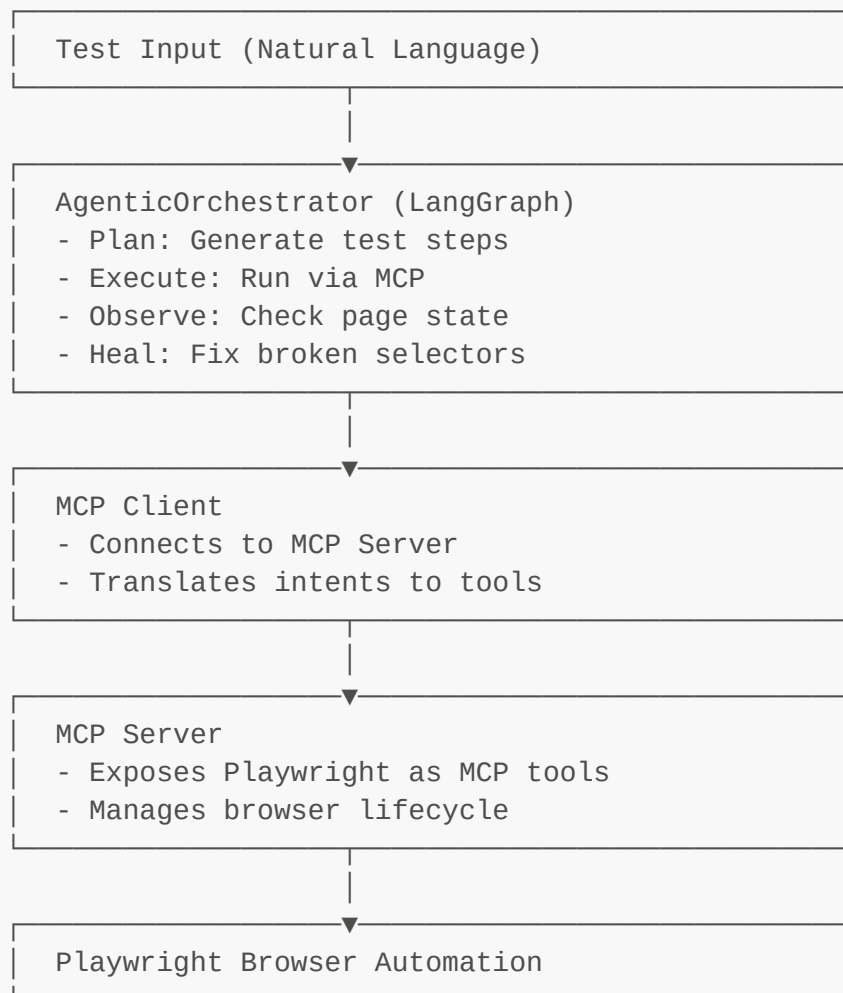
```
import { test, expect } from '@playwright/test';
import { SelfHealingManager } from '../src/self-healing/healing-
manager';

test('with self-healing', async ({ page }) => {
  const healer = new SelfHealingManager(page);

  await page.goto('https://example.com');

  // Automatically heals if selector changes
  await healer.clickWithHealing('Login button');
  await healer.fillWithHealing('Username field', 'test_user');
  await healer.verifyWithHealing('Welcome message');
});
```

Architecture Overview



Key Features

1. Natural Language Test Authoring

Instead of:

```
await page.goto('https://example.com');
await page.getByRole('button', { name: 'Login' }).click();
await page.fill('#username', 'test_user');
```

Write:

```
await agent.execute(`
  1. Go to example.com
  2. Click login button
  3. Fill username with test_user
`);
```

2. Self-Healing Tests

When elements change, AI automatically finds new selectors:

```
// Original: <button id="submit-btn">Submit</button>
// Changed:  <button id="submit-button">Submit</button>

// Traditional test: BREAKS ✕
// Self-healing test: ADAPTS ✔
await healer.clickWithHealing('Submit button');
```

3. Intelligent Test Data Generation

```
// Quick faker-based data
const user = testDataGenerator.generateBasicData('user');

// Contextual AI-generated data
const userData = await testDataGenerator.generateContextualData(
  'user',
  'E-commerce checkout for premium membership'
);

// Pre-built profiles
const profile = testDataGenerator.generateUserProfile();
const checkout = testDataGenerator.generateCheckoutData();
```

4. Cost Optimization

- **Prompt Caching:** Reuses common context (90% cost reduction)

- **Multi-Model Strategy:** Uses cheap models for simple tasks
- **Intelligent Fallbacks:** Traditional selectors first, AI only when needed

📱 Example Use Cases

E-commerce Testing

```
test('complete purchase flow', async ({ agent }) => {
  await agent.execute(`
    1. Navigate to shop.example.com
    2. Search for "laptop"
    3. Filter by price $500-$1500
    4. Add first item to cart
    5. Apply discount code "SAVE20"
    6. Proceed to checkout
    7. Fill shipping with realistic data
    8. Complete purchase with test card
    9. Verify order confirmation
  `);
});
```

Form Validation

```
test('registration form validation', async ({ agent }) => {
  await agent.execute(`
    1. Go to registration page
    2. Leave all fields empty
    3. Click register
    4. Verify error messages appear
    5. Fill valid data for all fields
    6. Submit form
    7. Verify success message
  `);
});
```

Cross-Browser Testing

```
test.use({ browserName: 'firefox' });
test.use({ browserName: 'webkit' });

// Tests automatically adapt to browser differences
```

📊 Monitoring & Reports

View Test Results

```
# Playwright HTML report
npx playwright show-report

# Allure report (detailed)
npx allure generate allure-results
npx allure open
```

Token Usage Tracking

All LLM calls are tracked:

- Tokens per test
- Cost estimates
- Model usage breakdown

Check [/api/stats/tokens](#) endpoint for real-time data.

Self-Healing Audit

Review all auto-healed selectors:

- Original vs healed selector
- Confidence scores
- Approval status

Requires human review before merging to prevent masking bugs.

🔧 Debugging

Enable Debug Logging

```
LOG_LEVEL=debug
```

Run Tests in UI Mode

```
npm run test:ui
```

Debug Specific Test

```
npx playwright test --debug tests/my-test.spec.ts
```

View Traces

```
npx playwright show-trace trace.zip
```

🚢 Deployment

Docker

```
# Build and run
docker-compose up -d

# View logs
docker-compose logs -f app

# Stop
docker-compose down
```

Environment Variables (Production)

```
NODE_ENV=production
DATABASE_URL=postgresql://...
REDIS_URL=redis://...
ANTHROPIC_API_KEY=sk-...
ENABLE_SELF_HEALING=true
REQUIRE_HUMAN_APPROVAL=true
```

💰 Cost Estimation

Based on Claude 3.5 Sonnet with prompt caching:

PROF

Operation	Tokens	Cost per Run
Test Plan Generation	2,000-4,000	\$0.10-0.30
Self-Healing Attempt	1,000-3,000	\$0.05-0.15
Visual Analysis	3,000-8,000	\$0.20-0.50

Monthly estimate (1000 tests/day, 10% healing rate):

- Without optimization: \$5,000-10,000
- With prompt caching: \$500-2,000

🔧 Best Practices

1. **Start simple:** Use natural language for test planning, traditional Playwright for execution
2. **Enable caching:** Always use prompt caching in production

3. **Review healing:** Require human approval for auto-healed selectors
4. **Monitor costs:** Track token usage per test
5. **Use cheap models:** Data generation and simple tasks don't need expensive models
6. **Cache selectors:** Successful healing results are cached for reuse
7. **Test isolation:** Each test gets fresh browser context

Resources

- [Model Context Protocol Docs](#)
- [Playwright Documentation](#)
- [LangGraph Guide](#)
- [Claude API Reference](#)

Support

For issues or questions:

1. Check the logs in `logs/app.log`
2. Review test traces in `test-results/`
3. Inspect self-healing cache with `healer.getCacheStats()`

Next Steps

1. Run example tests: `npm test tests/example-tests.spec.ts`
2. Try self-healing demo: `npm test tests/self-healing-demo.spec.ts`
3. Open web interface: `open src/ui/index.html`
4. Write your first natural language test
5. Monitor results in Playwright report